COMSATS University Islamabad, Lahore Campus

COAL Lab Assignment 03

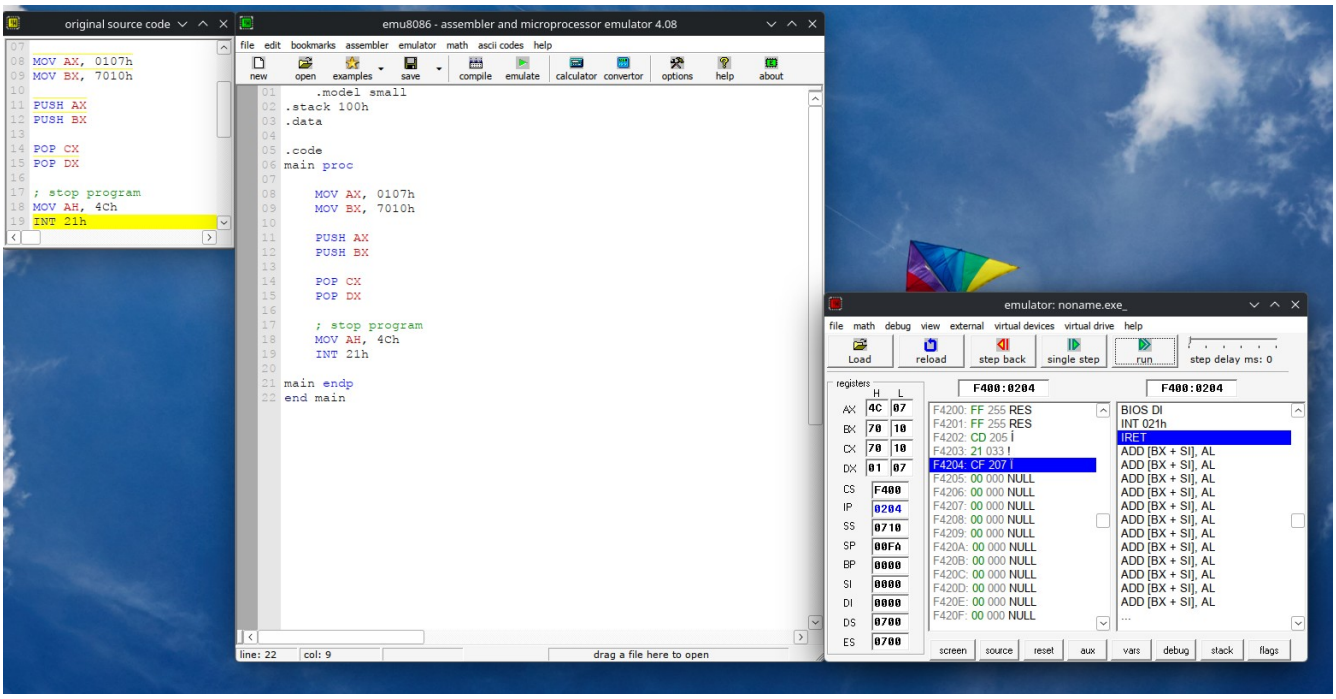
Submitted to: Sir Hamza Hamid

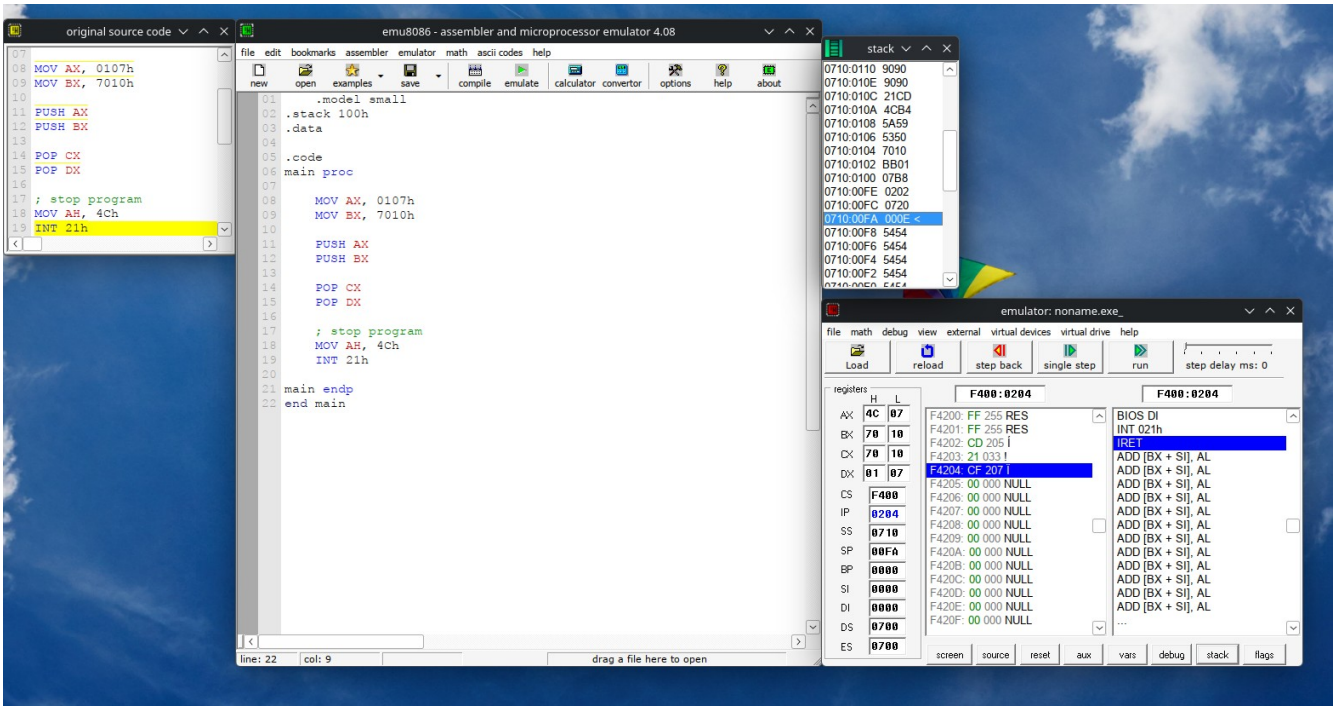
Submitted by: Samiullah Ashfaq

SP24-BSE-107-A

A. Stack Operations

```
.model small
.stack 100h
.data

.code
main proc

    mov ax, 0107h       ; RollNumber
    mov bx, 7010h       ; Reverse RollNumber

    push ax
    push bx

    pop cx
    pop dx

    mov ah, 4ch
    int 21h

main endp
end main
```

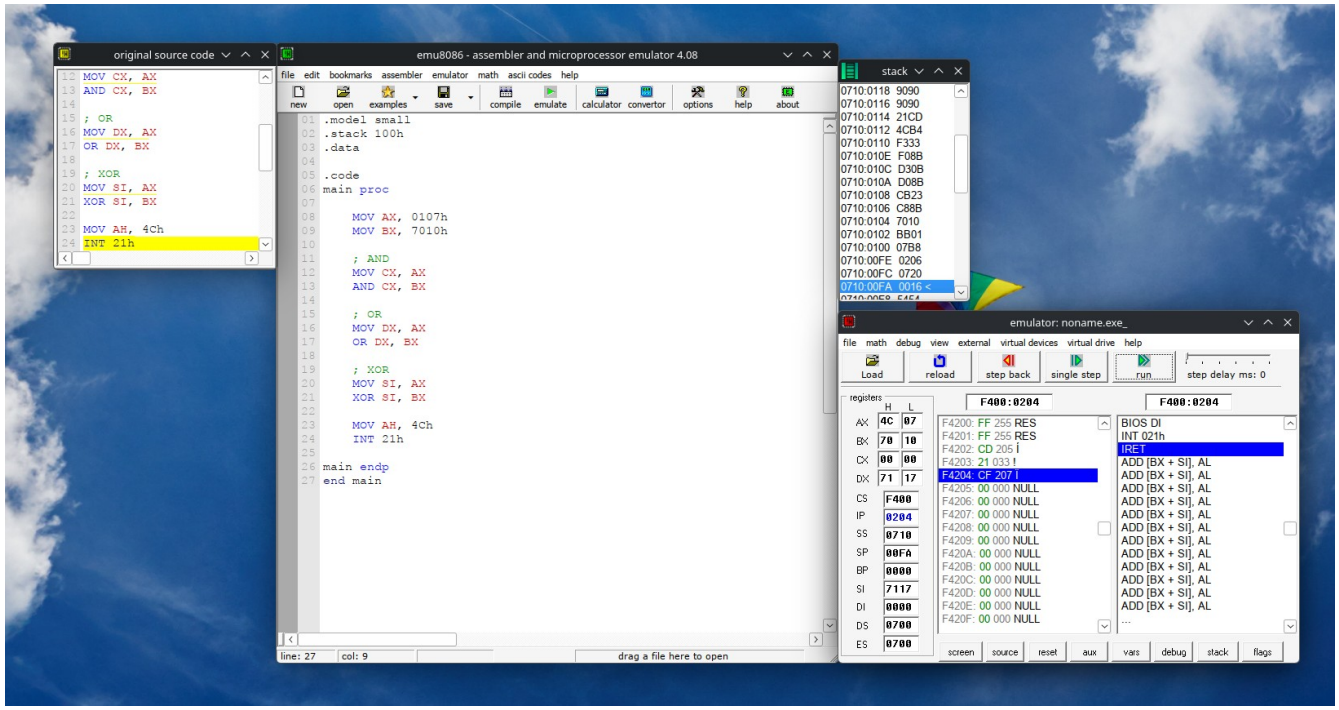




B. AND, OR, XOR

```
.model small
.stack 100h
.data

.code
main proc

    MOV AX, 0107h
    MOV BX, 7010h

    ; AND
    MOV CX, AX
    AND CX, BX

    ; OR
    MOV DX, AX
```

```
    OR DX, BX

    ; XOR
    MOV SI, AX
    XOR SI, BX

    MOV AH, 4Ch
    INT 21h

main endp
end main
```



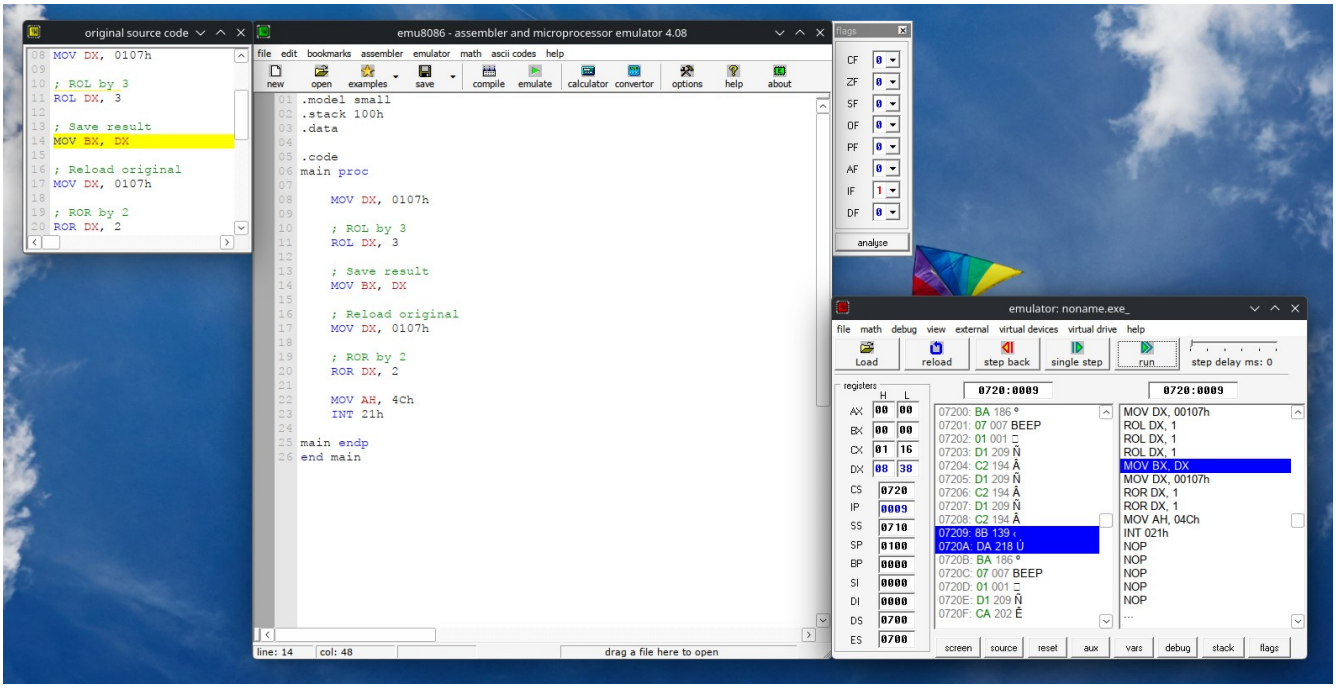
Results:

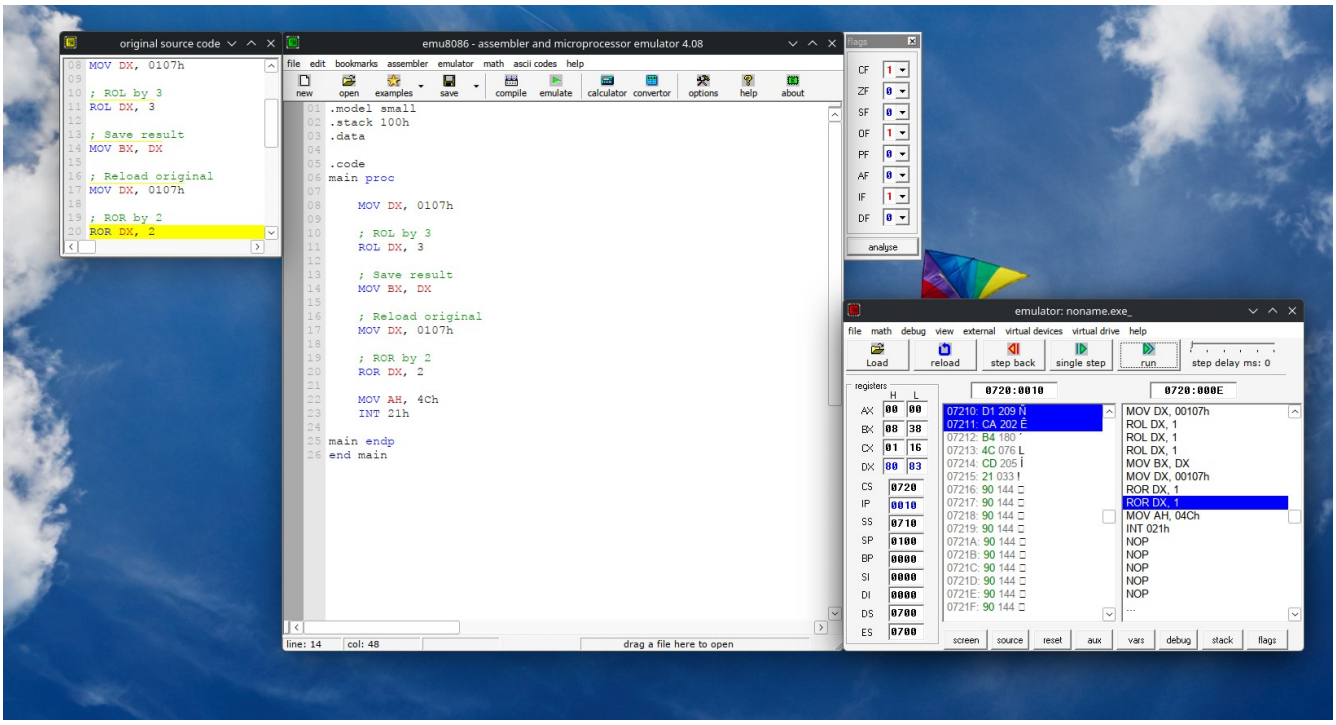
AX = 0107h = 0000 0001 0000 0111

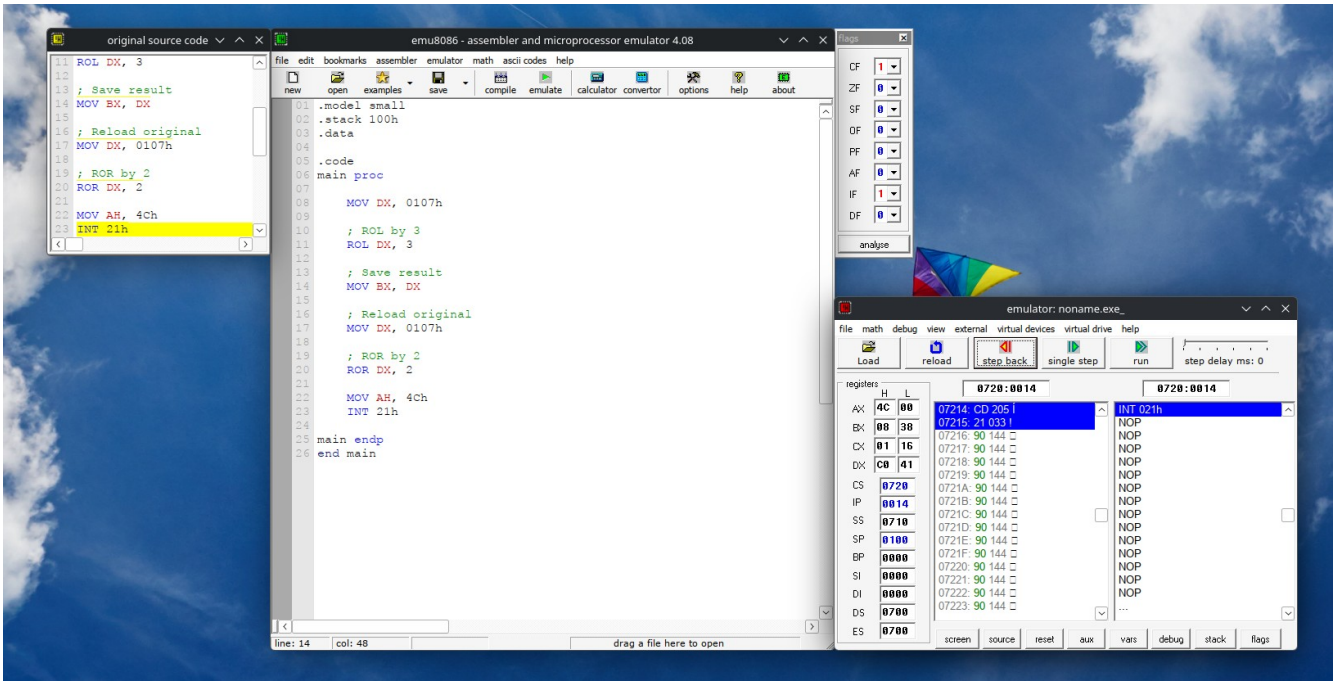
BX = 7010h = 0111 0000 0001 0000

C. Rotate

```
.model small
.stack 100h
.data

.code
main proc

    MOV DX, 0107h

    ; ROL by 3
    ROL DX, 3

    ; Save result
    MOV BX, DX

    ; Reload original
    MOV DX, 0107h

    ; ROR by 2
    ROR DX, 2

    MOV AH, 4Ch
    INT 21h

main endp
end main
```

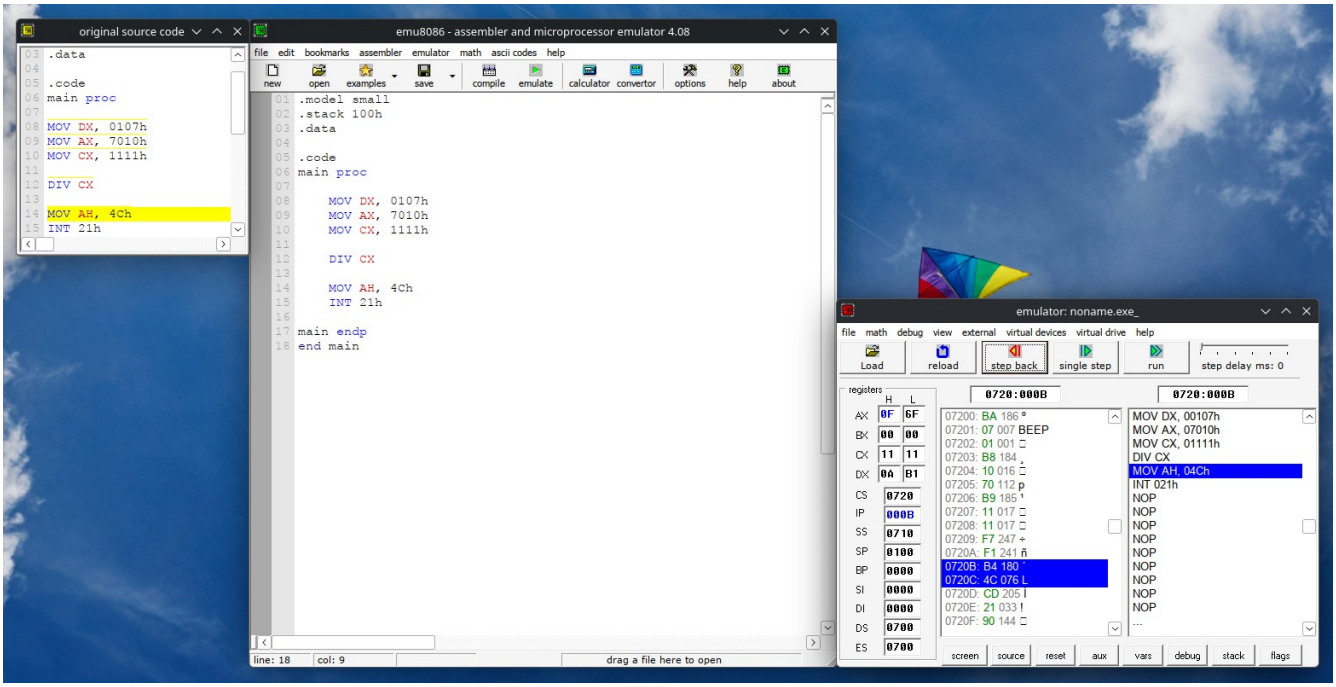

original source code
MOV DX, 0107h
; ROL by 3
ROL DX, 3
; Save result
MOV BX, DX
; Reload original
MOV DX, 0107h
; ROR by 2
ROR DX, 2
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about
.model small
.stack 100h
.data
.code
main proc
MOV DX, 0107h
; ROL by 3
ROL DX, 3
; Save result
MOV BX, DX
; Reload original
MOV DX, 0107h
; ROR by 2
ROR DX, 2
MOV AH, 4Ch
INT 21h
main endp
end main
line: 14
col: 48
drag a file here to open
flags
CF 0
ZF 0
SF 0
OF 0
PF 0
AF 0
IF 1
DF 0
analyse
emulator: noname.exe_
file math debug view external virtual devices virtual drive help
Load reload step back single step run step delay ms: 0
registers
H L
AX 00 00
BX 00 00
CX 01 16
DX 08 38
CS 0720
IP 0009
SS 0710
SP 0100
BP 0000
SI 0000
DI 0000
DS 0700
ES 0700
0720:0009
07200: BA 186 °
07201: 07 007 BEEP
07202: 01 001
07203: D1 209 Ñ
07204: C2 194 Â
07205: D1 209 Ñ
07206: C2 194 Â
07207: D1 209 Ñ
07208: C2 194 Â
07209: 8B 139
0720A: DA 218 Ú
0720B: BA 186 °
0720C: 07 007 BEEP
0720D: 01 001
0720E: D1 209 Ñ
0720F: CA 202 Ê
0720:0009
MOV DX, 00107h
ROL DX, 1
ROL DX, 1
ROL DX, 1
MOV BX, DX
MOV DX, 00107h
ROR DX, 1
ROR DX, 1
MOV AH, 04Ch
INT 021h
NOP
NOP
NOP
NOP
NOP
screen source reset aux vars debug stack flags


original source code
MOV DX, 0107h
; ROL by 3
ROL DX, 3
; Save result
MOV BX, DX
; Reload original
MOV DX, 0107h
; ROR by 2
ROR DX, 2
emu8086 - assembler and microprocessor emulator 4.08
file edit bookmarks assembler emulator math ascii codes help
new open examples save compile emulate calculator convertor options help about
.model small
.stack 100h
.data
.code
main proc
MOV DX, 0107h
; ROL by 3
ROL DX, 3
; Save result
MOV BX, DX
; Reload original
MOV DX, 0107h
; ROR by 2
ROR DX, 2
MOV AH, 4Ch
INT 21h
main endp
end main
line: 14
col: 48
drag a file here to open
flags
CF 1
ZF 0
SF 0
OF 1
PF 0
AF 0
IF 1
DF 0
analyse
emulator: noname.exe_
file math debug view external virtual devices virtual drive help
Load reload step back single step run step delay ms: 0
registers
H L
AX 00 00
BX 08 38
CX 01 16
DX 00 03
CS 0720
IP 0010
SS 0710
SP 0100
BP 0000
SI 0000
DI 0000
DS 0700
ES 0700
0720:0010
07210: D1 209 Ñ
07211: CA 202 Ê
07212: B4 180
07213: 4C 076 L
07214: CD 205 Í
07215: 21 033 !
07216: 90 144
07217: 90 144
07218: 90 144
07219: 90 144
0721A: 90 144
0721B: 90 144
0721C: 90 144
0721D: 90 144
0721E: 90 144
0721F: 90 144
0720:000E
MOV DX, 00107h
ROL DX, 1
ROL DX, 1
ROL DX, 1
MOV BX, DX
MOV DX, 00107h
ROR DX, 1
ROR DX, 1
MOV AH, 04Ch
INT 021h
NOP
NOP
NOP
NOP
NOP
screen source reset aux vars debug stack flags



D. Division

```
.model small
.stack 100h
.data

.code
main proc

    MOV DX, 0107h
    MOV AX, 7010h
    MOV CX, 1111h

    DIV CX

    MOV AH, 4Ch
    INT 21h

main endp
end main
```



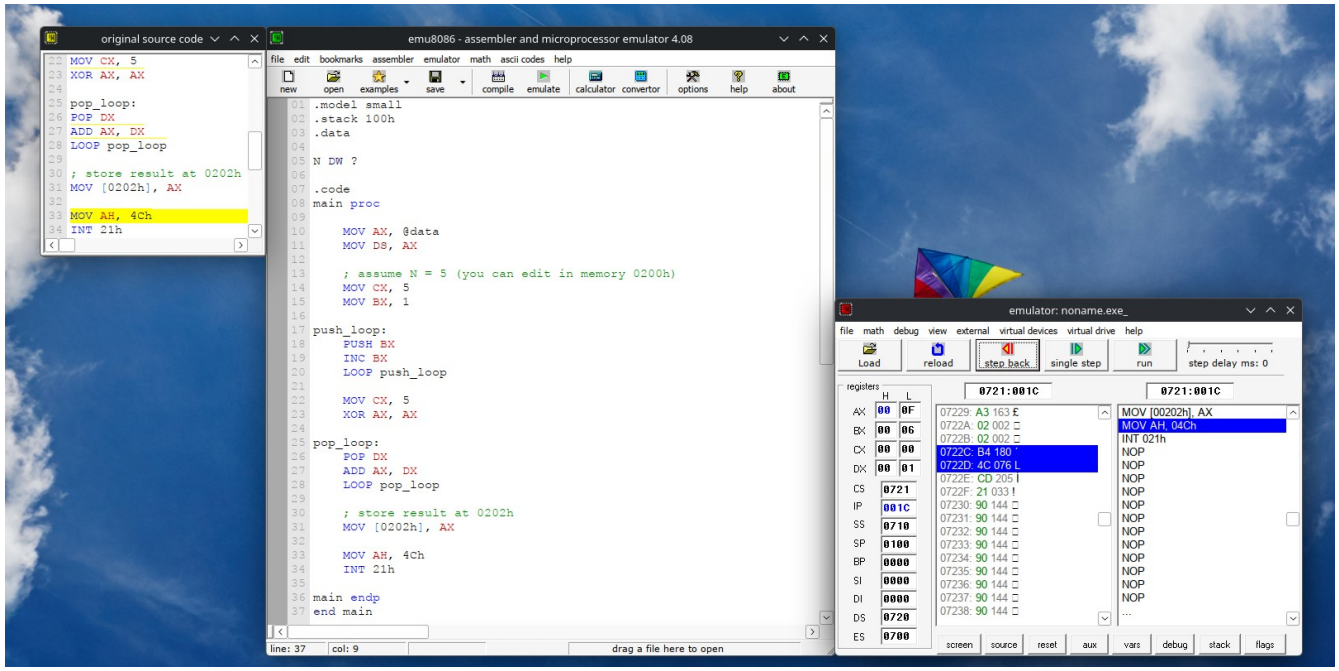
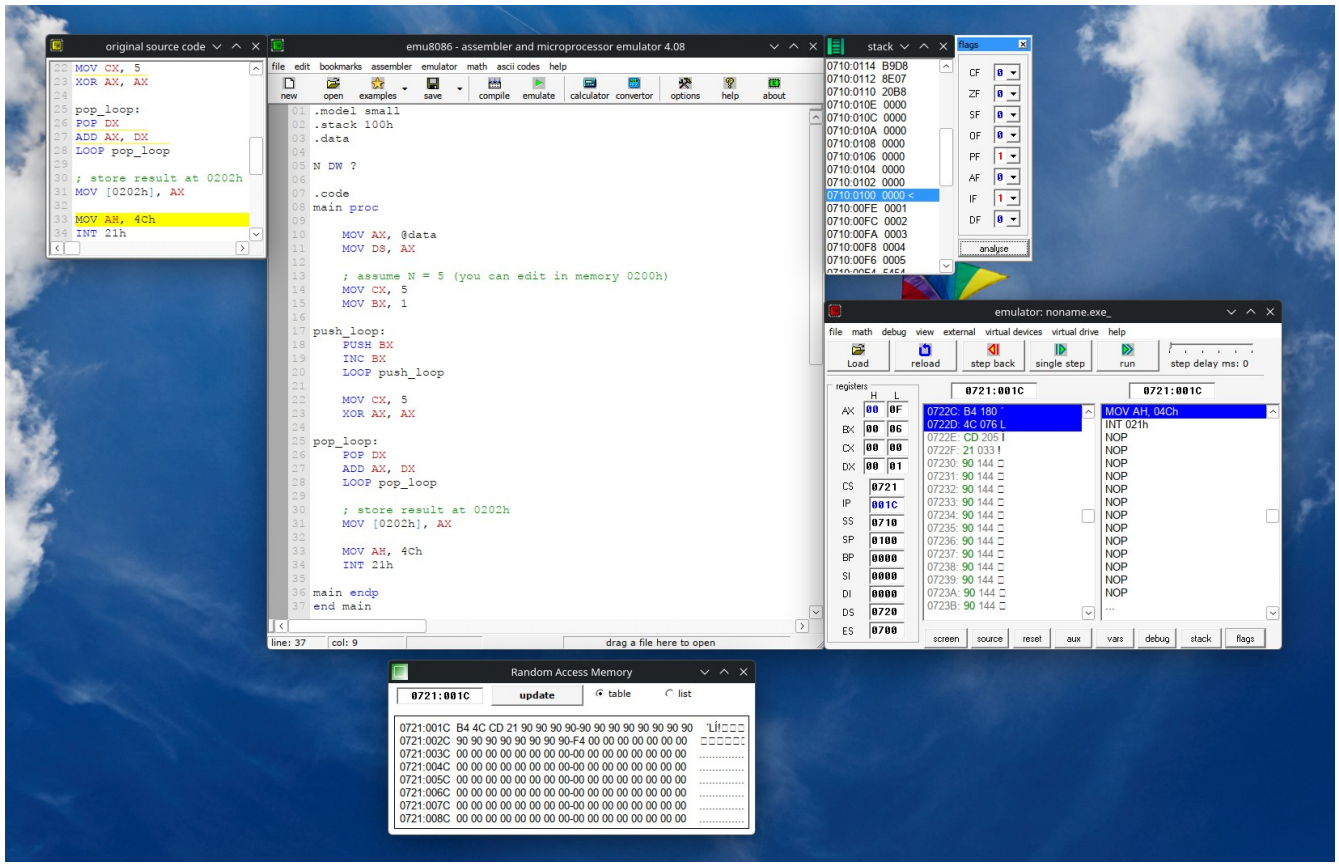
Result:

- **Quotient = 3951 = 0F6Fh**
- **Remainder = 2125 = 084Dh**

E. Sum using Stack + Loop

```
.model small

.stack 100h

.data

.code

main proc


    MOV AX, @data
```

```
    MOV DS, AX

    ; assume N = 5
    MOV CX, 5
    MOV BX, 1

push_loop:
    PUSH BX
    INC BX
    LOOP push_loop

    MOV CX, 5
    XOR AX, AX

pop_loop:
    POP DX
    ADD AX, DX
    LOOP pop_loop

    ; store result at 0202h
    MOV [0202h], AX

    MOV AH, 4Ch
    INT 21h
```

main endp

end main